

APB Subsystem — UVM Verification Plan

apb_master · apb_decoder · apb_mux · apb_slave · apb_registerfile · apb_top

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
Reset Behaviour				
APB_1	When PRESETn is asserted (low), all top-level outputs (done, error) must be deasserted and held at their default inactive state for the entire duration of the reset.	Directed reset applied at simulation start. Randomised via constraint c_PRESETn so that PRESETn=0 occurs ~5% of the time across the run.	cp_reset (CovGroupSys): bins in_reset={1'b0} and out_of_reset={1'b1} ensure both reset states are exercised.	Assertion APB_22 (PRESETn_clear in apb_top): !PRESETn -> (done==0 && error==0) checked every posedge PCLK.
APB_2	On the falling edge of PRESETn, APB bus signals PSEL and PENABLE must deassert immediately in the next cycle. No in-flight transfer shall survive a reset event.	Reset injected mid-transfer by enabling c_PRESETn without c_no_reset. Driver pulses PRESETn low for one negedge PCLK then releases.	—	Assertion a_reset_state (apb_master_sva): \$fell(PRESETn) => (!PSEL && !PENABLE && !done && !error).
Write Transactions				
APB_3	A write transaction (rw=1) must progress through IDLE→SETUP→ACCESS and complete with done=1. The master must drive PWRITE=1, PSEL=1 in SETUP, and PENABLE=1 in ACCESS.	Randomised via trans_type==APB_WRITE. Constraint c_write_cmd forces write-only sequences when needed. addr built by c_addr_valid targeting slave0–slave3.	cp_trans_type bin write (CovGroupTxn). cross_type_slave ensures every slave receives at least one write.	Scoreboard compares write data against register-file golden model. Assertion APB_26 (finish_transaction): start -> ##[1:\$] (done error).
APB_4	PWDATA and PADDR must remain stable from PSEL assertion through the end of ACCESS (PREADY high). Any change before PREADY constitutes an APB protocol violation.	Driver holds addr/wdata/byte_en steady across the full transfer window. start is pulsed for exactly one cycle; the RTL registers the values internally.	—	Assertions APB_27 (write_data_stable): (PSEL&&PENABLE&&PWRITE&&!PREADY) => \$stable(PWDATA). APB_28 (address_stable): (PSEL&&PENABLE&&!PREADY) => \$stable(PADDR).
APB_5	PSTRB (byte-enable strobes) must remain stable throughout ACCESS and must not be all-zero on any write transaction.	Constraint c_byte_en_valid prevents byte_en==4'b0000 on writes. Driver maps byte_en to PSTRB directly.	cp_byte_en (CovGroupTxn): bins byte0, byte1, byte2, byte3, halfword_lo, halfword_hi, fullword, other_mix (default).	Assertion APB_29 (pstrb_stable): (PSEL&&PENABLE&&!PREADY) => \$stable(PSTRB). Scoreboard checks per-byte write correctness using wstrb.

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
APB_6	For each asserted PSTRB bit the corresponding register byte must update to the new value. Bytes with PSTRB=0 must remain unchanged (byte-lane isolation).	All single-byte, half-word, and full-word byte_en patterns generated across the run using c_byte_en_valid.	cp_byte_en — all named bins must be hit (CovGroupTxn).	Assertions a_byte_written / a_byte_held (apb_regfile_sva): genvar loop per byte lane checks update vs. hold one cycle after write_enable.
Read Transactions				
APB_7	A read transaction (rw=0) must complete with the correct register data on rdata and done=1. PWRITE must be 0 throughout the transfer.	Constraint c_read_cmd forces read-only sequences when needed. Addr generated by c_addr_valid after a preceding write so rdata can be verified against a known value.	cp_trans_type bin read (CovGroupTxn). cross_type_slave ensures every slave is read at least once.	Scoreboard compares returned rdata against rdata_ref from the golden model written on the preceding write to the same address.
APB_8	Read data (PRDATA routed to rdata) must reflect the last value written to that address. Reading a never-written register must return 0x00000000 (reset value).	Directed write-then-read sequence for every register offset (addr[5:2]=0–15) in each slave. Scoreboard tracks per-address expected values from reset.	cp_reg_sel bins reg_idx[16] (CovGroupTxn): all 16 register offsets within each slave must be accessed.	Assertion a_read_reflects_mem (apb_regfile_sva): read_enable -> (rdata==mem[addr]). Scoreboard final comparison at end of test.
Address Decode & Slave Selection				
APB_9	The decoder must assert exactly one PSELx for addresses 0x0000–0x3FFF (one of PSEL0–PSEL3). At most one slave may be selected at any time (\$onehot0).	Randomised addr covers all four slave windows via c_slave_reg_range and c_addr_valid. Boundary addresses (0x0FFF, 0x1000, 0x1FFF, 0x2000, etc.) hit by directed tests.	cp_slave bins slave0–slave3 and unmapped (CovGroupTxn). cross_type_slave and cross_slave_offset include all four slaves.	Assertion APB_31 (one_slave_only): \$onehot0({PSEL0,PSEL1,PSEL2,PSEL3}) every cycle. Decoder-level assertions a_onehot0_select and a_full_decode (apb_decoder_sva).
APB_10	For addresses 0x4000–0xFFFF (fully unmapped), the decoder must assert PSEL_ERR and deassert all PSELx. The mux must return PSLVERR=1, PREADY=1, PRDATA=0x00000000.	Constraint c_addr_invalid_unmapped: invalid_addr==1 && invalid_kind==1 → addr inside [0x4000:0xFFFF], generated ~5% of the time via c_invalid_addr_dist.	cp_slave bin unmapped (addr[15:12] inside [4:F]). cp_pslverr bin error ensures PSLVERR=1 path is covered.	Assertion APB_30 (invalid_address): PADDR>=0x4000 -> PSLVERR. Assertion APB_32 (invalid_decode): PSEL_ERR -> !(PSEL0 PSEL1 PSEL2 PSEL3). Assertion a_route_err (apb_mux_sva).
APB_11	For addresses within a valid slave window but beyond offset 0x3C (in-window-but-invalid), the slave	Constraint c_addr_invalid_in_window: invalid_addr==1 && invalid_kind==0 → addr in [(slave_base+0x40):(slave_base+0xFFF)].	cross_slave_offset (CovGroupTxn): cp_slave × cp_offset_valid, with ignore_bins	Assertion a_oob_errors (apb_slave_sva): (PSEL&&PENABLE&&PADDR[11:0]>0x3C) => PSLVERR. Scoreboard checks pslverr_ref matches received pslverr.

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
	must respond with PSLVERR=1 and PREADY=1 — not hang the bus.	pre_randomize() prevents two consecutive invalid addresses.	unmapped_ignore excluding fully-unmapped case.	
APB Protocol Compliance — FSM & Bus Sequencing				
APB_12	The master FSM must sequence correctly: SETUP phase has PSEL=1 & PENABLE=0 exactly one cycle after start asserts, followed by ACCESS phase with PSEL=1 & PENABLE=1.	Every transaction generated by the driver exercises the SETUP→ACCESS path. start is pulsed for exactly one negedge PCLK then deasserted, leaving the FSM to step through states.	—	Assertion APB_23 (setup_signals): start => (PSEL==1 && !PENABLE). Assertion APB_24 (access_signals): (PSEL&&!PENABLE) => (PSEL&&PENABLE). Assertions a_psel_setup_high / a_penable_setup_low (apb_master_sva).
APB_13	The master must wait for PREADY before completing ACCESS. It must never abandon a transfer or return to IDLE while PREADY is still low (no early termination).	Slave always responds with PREADY after one wait state (registered output, fixed latency). Directed back-to-back test verifies done never asserts while PREADY is still low.	—	Assertion APB_25 (wait_slave): (PSEL&&PENABLE) -> ##[1:\$] PREADY. Assertion a_pready_latency (apb_slave_sva): (PSEL&&PENABLE) => PREADY (one-wait-state contract pinned down).
APB_14	PENABLE must deassert on the cycle after PREADY goes high (transfer completion). It must not carry over into the SETUP phase of the next back-to-back transfer.	Back-to-back transaction sequences generated by keeping start asserted immediately after done asserts.	—	Assertion a_penable_one_cycle (apb_master_sva): PENABLE => !PENABLE PREADY. Combined with a_penable_setup_low to cover the full PENABLE lifecycle.
APB_15	done and error must be mutually exclusive — they must never assert in the same clock cycle.	Randomised mix of valid, in-window-invalid, and fully-unmapped transactions exercises both done and error paths. Reset mid-transfer also exercised.	—	Assertion a_done_error_exclusive (apb_top_sva): !(done && error). Assertion a_done_one_cycle (apb_master_sva): done => !done. Assertion a_error_causality: error -> \$past(PREADY&&PSLVERR).
Signal Stability During Transfer				
APB_16	PWDATA must remain stable on every cycle of the ACCESS phase where PWRITE=1 and PREADY=0. A change mid-transfer is a protocol error.	Driver does not re-randomise wdata between SETUP and ACCESS. Directed test forces a long-latency slave to create multi-cycle ACCESS windows.	—	Assertion APB_27 (write_data_stable in apb_top): (PSEL&&PENABLE&&PWRITE&&!PREADY) => \$stable(PWDATA).
APB_17	PADDR must remain stable throughout the entire SETUP and ACCESS phases until	addr fixed at the start of each transaction by the driver; not re-driven until done asserts.	—	Assertion APB_28 (address_stable in apb_top): (PSEL&&PENABLE&&!PREADY) => \$stable(PADDR). Assertion a_addr_stable (apb_master_sva)

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
	PREADY completes the transfer.			provides the same check inside the master module.
APB_18	PSTRB must remain stable throughout ACCESS while PREADY is low. (Covered by APB_29 already in RTL, listed here for traceability.)	Same as APB_5 stimulus: byte_en is fixed at the start of each transaction.	—	Assertion APB_29 (pstrb_stable in apb_top): disable iff(!PRESETn); (PSEL&&PENABLE&&!PREADY) => \$stable(PSTRB).
Decoder & MUX Response Assertions (APB_30–APB_32)				
APB_19	Any address >= 0x4000 presented on PADDR must immediately produce PSLVERR=1 from the mux (PSEL_ERR path). There must be no delay and no silent discard.	c_addr_invalid_unmapped generates unmapped addresses ~5% of the time. Directed test exercises 0x4000 (boundary), 0x8000, and 0xFFFF.	cp_slave bin unmapped, cp_pslverr bin error in CovGroupTxn.	Assertion APB_30 (invalid_address in apb_top): (PADDR >= 16'h4000) -> PSLVERR checked every posedge PCLK.
APB_20	At most one PSELx (PSEL0–PSEL3) may be asserted at any time. Multiple slaves cannot be simultaneously selected — this would corrupt both write data and read data.	All four slave address ranges exercised including boundary values. This condition must hold for every transaction and every idle cycle.	—	Assertion APB_31 (one_slave_only in apb_top): \$onehot0({PSEL0,PSEL1,PSEL2,PSEL3}) checked every posedge PCLK across the full simulation.
APB_21	When PSEL_ERR is asserted (unmapped address), all four PSELx lines (PSEL0–PSEL3) must be deasserted. PSEL_ERR and any PSELx are mutually exclusive.	Same stimulus as APB_10 and APB_19: unmapped addresses generated by c_addr_invalid_unmapped.	—	Assertion APB_32 (invalid_decode in apb_top): PSEL_ERR -> !(PSEL0 PSEL1 PSEL2 PSEL3) checked every posedge PCLK.
Functional Coverage Closure				
APB_22	CovGroupSys must reach 100%: both reset states (PRESETn=0 and PRESETn=1) must be observed during simulation.	c_PRESETn: PRESETn dist {1:=95, 0:=5}. Directed reset at sim start guarantees in_reset bin is hit on the very first transaction.	cp_reset — bins: in_reset (PRESETn=0), out_of_reset (PRESETn=1).	—
APB_23	cp_trans_type must cover both write and read bins. Every slave (slave0–slave3) must	c_write_cmd and c_read_cmd soft constraints used in alternating sequence tests. c_slave_reg_range distributes across all four slaves.	cp_trans_type: bins write, read. cross_type_slave: 2 trans_types × 5 slave	—

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
	be accessed with both transaction types.		bins = 10 cells (all must close).	
APB_24	cp_slave must cover all five bins: slave0, slave1, slave2, slave3, and unmapped. The unmapped bin requires invalid_addr==1 && invalid_kind==1.	c_slave_reg_range selects slave 0–3. c_addr_invalid_unmapped generates the unmapped case ~5% of the time.	cp_slave (CovGroupTxn): bins slave0 (addr[15:12]==0), slave1 (==1), slave2 (==2), slave3 (==3), unmapped ([4:F]).	—
APB_25	cp_offset_valid must cover both valid offsets (addr[11:0] <= 0x3C) and invalid offsets (> 0x3C). Both address classes must appear in completed transactions.	c_addr_valid produces valid offsets. c_addr_invalid_in_window produces in-window-invalid offsets. pre_randomize() prevents two consecutive invalid addresses.	cp_offset_valid: bins valid_offset (addr[11:0]<=0x3C=1), invalid_offset (=0).	—
APB_26	cp_reg_sel must close all 16 reg_idx bins. All 16 register offsets (addr[5:2]=0–15) within each slave must be accessed at least once across the run.	Directed write-then-read sweep for every register index in each of the four slaves (64 directed transactions total). Randomised run fills gaps.	cp_reg_sel: bins reg_idx[16] = {[0:15]} — 16 auto-named bins each representing one 4-byte-aligned register offset.	—
APB_27	cp_byte_en must close all 8 bins including other_mix (default). Every meaningful byte-strobe pattern must appear on a write transaction.	c_byte_en_valid allows all non-zero patterns. Directed tests exercise each named bin explicitly; randomised run fills other_mix.	cp_byte_en: bins byte0 (0001), byte1 (0010), byte2 (0100), byte3 (1000), halfword_lo (0011), halfword_hi (1100), fullword (1111), other_mix (default).	—
APB_28	cp_pslverr must cover both ok (PSLVERR=0) and error (PSLVERR=1) outcomes. The error bin requires either an unmapped or an in-window-invalid address.	Both valid and invalid transactions mixed across the run. c_addr_invalid_unmapped and c_addr_invalid_in_window generate the error cases.	cp_pslverr: bins error={1'b1}, ok={1'b0}.	—
APB_29	cross_type_slave (APB_19) must fully close: every combination of transaction type (write/read) × slave target (slave0–slave3	Long regression alternates write and read transactions directed at each slave. Unmapped writes and reads both exercised via invalid_addr flag.	cross_type_slave: 2 × 5 = 10 cells. All 10 must close including (write × unmapped) and (read × unmapped).	—

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
	+ unmapped) must be observed.			
APB_30	cross_type_offset (APB_20) must fully close: both transaction types must be exercised against both valid and invalid offset ranges.	c_read_cmd + c_addr_invalid_in_window generates read-to-invalid-offset. c_write_cmd + c_addr_valid generates the valid-offset write case.	cross_type_offset: 2 trans_types × 2 offset bins = 4 cells. All 4 must close.	—
APB_31	cross_slave_offset (APB_21) must fully close: each of the four valid slaves must be accessed with both valid and invalid offsets. The unmapped × valid-offset combination is architecturally excluded via ignore_bins unmapped_ignore.	Four directed in-window-invalid tests (one per slave). Randomised run fills the valid-offset bins across all slaves.	cross_slave_offset: 5 slave_bins × 2 offset bins = 10 cells, minus 2 (ignore_bins unmapped_ignore on valid_offset side) = 8 cells to close.	—
APB_32	CovGroupTxn sampling is gated on PRESETn==1 && done==1 so only completed non-reset transactions count toward coverage. This ensures 100% coverage reflects real protocol traffic, not reset artefacts.	c_no_reset soft constraint keeps PRESETn=1 during coverage-collection phases. done is only asserted by the master FSM on a real completed transfer.	Sampling gate: if(seq_item.PRESETn==1 && seq_item.done) CovGroupTxn.sample(). All CovGroupTxn bins gated by this condition.	Scoreboard cross-checks every sampled transaction against the golden model to prevent incorrect transactions from inflating coverage numbers.

Labels APB_1–APB_32 align with constraint (c_*), coverpoint (cp_*), cross, and assertion identifiers in the RTL and UVM environment.